AMERICAN UNIVERSITY OF BEIRUT
Faculty of Arts and Sciences
Department of Computer Science
CMPS 240 – Operating Systems
Spring 2025 – 2026

Assignment 2: Multithreading, Pipes, Synchronization and Scheduling

IMPORTANT GUIDELINES

1. Submit the assignment on Moodle.
2. Zip and attach the `.c` files for all problems, the file name should be: `YourName_Assignment2.zip` and each `.c` file `ExerciseX.c` where `X` is the exercise's number.

Exercise 1: Concurrent Multi-Account Banking System with Synchronized Operations and Logging.

Implement a multithreaded banking system using threads and semaphores to simulate multiple users performing simultaneous operations on multiple shared accounts. You should avoid race conditions on all shared data.

- The system contains `N` bank accounts (e.g., `N = 5`), each initialized with a random balance between 500 and 1500.
- There are M user threads (e.g., `M = 10`), each performing a series of random transactions.
- Each transaction could be a:
    1. Deposit transaction that adds a random amount (50–300) to an account.
    2. Withdraw transaction that subtracts a random amount (50–300) if there's enough balance.
- Each account must be protected by its own semaphore, so threads can safely operate on different accounts in parallel.
- A global string array log that is shared between all threads records all successful and failed transactions:
    - Note that the log array access should be also protected by a semaphore.

Each user thread repeatedly:

1. Chooses a random transaction type.
2. Selects one random account ID.
3. Chooses a random transaction amount.
4. Performs the transaction (under semaphore protection).
5. Logs the result to the global log array and prints it to the console.

Each thread should perform at least 5–10 transactions.
At the end, the main thread should print:

- The **final balance of each account**
- The **log that** contains all records

Submission Requirements

- Source code file (.c)
- Screenshot of the program output (showing your name in the output)

A sample application that uses semaphores (*i.e.*, `sem_wait`, `sem_post`, `sem_init`, and `sem_destroy`) follows:

In the below code, the header line of `sem_init` is as follows:

```
int sem_init(sem_t *sem, int pshared, unsigned value);
```

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_ITERATIONS 100000

int counter = 0;            // Shared variable
sem_t sem;                  // Shared Semaphore

void* increment(void* arg) {
    for (int i = 0; i < NUM_ITERATIONS; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    // Initialize semaphore with value 1
    sem_init(&sem, 0, 1);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem);

    printf("Final counter value: %d\n", counter);
    return 0;
}
```

Exercise 2: Pipes and Threads.

In this exercise, it is required to write a program that counts the number of occurrences of a word in a file using threads and pipes. The program reads the name of the text file, the word to search for, and the number of threads as command line arguments. It will then open the file and create two pipes:

1- The lines pipe in which the master thread writes the lines it reads from the file.
2- The counts pipe in which the threads write the number of occurrences of the words after they finish counting.

Then the main thread creates the threads, which, in turn, keep reading from the lines pipe and searching for the word and incrementing their respective counters. When the pipe is no longer written into, each thread writes its count into the pipe. When the main thread finishes reading the file and writing its lines into the lines pipe, it calculates the sum of all the counts that were written to the counts pipe.
One approach that can be adopted for indicating whether the other thread finished writing into the pipe consists of using `while(read(...) > 0)` where a `read()` operation made on an empty and closed pipe will return `0` and a read operation on an empty but not closed pipe is a blocking operation that will make the calling thread wait until something is available to read. To promote your familiarity and understanding of file descriptor tables, you will be required to use this method for input termination test. Therefore, a couple of important notes are due:

1- All threads share exactly the same descriptor table. A `close(f)` call will close `f` for all threads.
2- A call to `read()` will return `0` if *all file-descriptor entries* referring to the same pipe end are closed.
3- To use `close()`, the pipe descriptors must be duplicated using the `dup()` system call.

Consequently, for each created thread, use `dup()` to assign it duplicate descriptors of the pipe ends:

1- A duplicate for the reading end of `lines_pipe`
2- A duplicate for the writing end of the `counts_pipe`.

Pass these as arguments to each thread. For testing purposes, print these descriptors inside the threads' function. The output to expect is:

```
From inside the thread: read from lines_pipe at: 8 and write to counts_pipe at 9
From inside the thread: read from lines_pipe at: 10 and write to counts_pipe at 11
From inside the thread: read from lines_pipe at: 12 and write to counts_pipe at 13
⋮
```

The descriptor table of the process will look as follows:

0	`stdin`
1	`stdout`
2	`stderr`
3	Input text file
4	`lines_pipe[0]`
5	`lines_pipe[1]`
6	`counts_pipe[0]`
7	`counts_pipe[1]`
8	duplicate of `lines_pipe[0]`
9	duplicate of `counts_pipe[1]`
10	Another duplicate of `lines_pipe[0]`
11	Another duplicate of `counts_pipe[1]`
12	Another duplicate of `lines_pipe[0]`
13	Another duplicate of `counts_pipe[1]`
⋮	

Exercise 3: Synchronization.

1- Write a program that uses `pthreads` and POSIX semaphores to simulate the creation of three types of threads: producers, consumers, and readers. The types of threads are created randomly with higher likeliness for readers. The classical reader-writer solution with priority for readers is used if the buffer is neither full nor empty. If the buffer is full, then consumers must not be made to wait by readers. If the buffer is empty, then producers must not be made to wait by readers. The code must not contain any busy waiting. All waiting must be handled by semaphores.

2- Assume a hypothetical scenario for execution suggesting a sequence of arrival of processes of the three types. Demonstrate how the values of the semaphores change and the queues are populated throughout the execution. Include a case where the buffer is either full or empty and readers are made to wait until the buffer state changes.

The following is an example with the first solution of readers and writers.
Assume arrival of the following sequence: `R1`, `R2`, `W1`, `W2`, `R3`, `R4` at time `0`. There are two semaphores `wrt` and `mutex` both initialized to `1`.

```
// Writer
while (TRUE) {
    wait(wrt);

    writing is performed

    signal(wrt);
}
```

```
// Reader
while (TRUE) {
    wait(mutex);
    readcount++;
    if(readcount == 1)
        wait(wrt);
    signal(mutex);

    reading is performed

    wait(mutex);
    readcount--;
    if(readcount == 0)
        signal(wrt);
    signal(mutex);
}
```

Arrival	`readcount`	`mutex` value	Waiting at `mutex` queue	`wrt` value	Waiting at `wrt` queue	Operating in data
`R1`	`1`	`1`	`--`	`0`	`--`	`R1`
`R2`	`2`	`1`	`--`	`0`	`--`	`R1 R2`
`W1`	`2`	`1`	`--`	`0`	`W1`	`R1 R2`
`W2`	`2`	`1`	`--`	`0`	`W1, W2`	`R1 R2`
`R3`	`3`	`1`	`--`	`0`	`W1, W2`	`R1 R2 R3`
`R4`	`4`	`1`	`--`	`0`	`W1, W2`	`R1 R2 R3 R4`
Readers finish one by one decrementing `readcount` until it is `0`:						
`--`	`0`	`1`	`--`	`0`	`W2`	`W1`
`W1` finishes						
`--`	`0`	`1`	`--`	`0`	`--`	`W2`
W2 finishes						
`--`	`0`	`1`	`--`	`1`	`--`	`--`

You will also have to make additional assumptions regarding buffer being full or empty (for the execution scenario let the buffer be small enough) and perhaps reader, producer, and consumer execution times. Therefore, your table can be time dependent such as:

`time`	`0`	`1`	`2`	`3`	`4`	`5`	...
arrival							
`wrt`							
queue							
`mutex`							
operating							
⋮							

Exercise 4: Scheduling.

Consider the following set of processes arriving at time $t = 0$

Process	Arrival	Burst Time (ms)
P_1	0	10
P_2	2	3
P_3	3	5
P_4	7	6
P_5	8	4

1. Compute the average waiting time under RR with $q = 4$. Show all the details that justify your answer.
2. How long should the context switching interval be, if all context switches equally consume 5% of the CPU time under RR. Show computation details for credit.

Exercise 5: Processes V.S. Threads

Answer the following questions:

1. What does it mean when we say that a process has a private address space?
2. What are the advantages and disadvantages of a private address space?
3. What programming directives have we used to circumvent address space privacy?
4. Threads are often called lightweight processes. In what sense are they processes? And why the lightweight distinction?
5. Threads are often called virtual processes as well. In what sense are threads an example of virtualization?
6. Threads running within the same process all share the same address space. What are the advantages and disadvantages of allowing threads to access pretty much all of virtual memory?

Now, each thread within a larger process is given a thread id, also called a `tid`. In fact, the thread id concept is just an extension of the process id. For singly threaded processes, the `pid` and the main thread's `tid` are precisely the same. If a process with `pid=12345` creates three additional threads beyond the main thread (and no other processes or threads within other processes are created), then the `tid` of the main thread would be `12345`, and the thread ids of the three other threads would be `12346`, `12347`, and `12348`. In this context, answer the below questions:

7. What are the advantages of leveraging the `pid` abstraction for thread ids?
8. What happens if you pass a thread id that isn't a process id to `waitpid()`?
9. What happens if you pass a thread id to `sched_setaffinity()`?
10. What are the advantages of requiring that a thread always be assigned to the same CPU?

Finally, in some situations, the decision to rely on multithreading instead of multiprocessing is dictated solely by whether the code to be run apart from the main thread is available in executable or in library form. But in other situations, we have a choice. In this context, answer the below questions:

11. Why might you prefer multithreading over multiprocessing if both are reasonably good options?
12. Why might you prefer multiprocessing over multithreading if both are reasonably good options?
13. What happens if a thread within a larger process calls `fork()`?
14. What happens if a thread within a larger process calls `execvp()`?

Exercise 6: Threading Short Answers

Answer the following questions:

1. Is `i--` thread safe? Why or why not?
2. What's the difference between a mutex and a semaphore with an initial value of 1? Can one be substituted for the other?
3. What is busy waiting? Is it ever a good idea? Does your answer to the good-idea question depend on the number of CPUs your multithreaded application has access to?
4. As it turns out, the semaphore's constructor allows a negative number to be passed in, as with semaphore `s(-11)`. Identify a scenario where `-11` might be a sensible initial value.

Exercise 7: XV6 Scheduling Function Modification

Assume XV6 is modified so that each process has an integer field `priority`, where a smaller value means a higher scheduling priority. The scheduler should always choose, among all `RUNNABLE` processes, the one with the smallest `priority` value. If multiple runnable processes have the same priority, the scheduler should select them in round-robin order among that priority level.

Part A: Understanding The Policy

1. In the modified scheduler, which of the processes in the table below should be selected first?

Process	State	Priority
P1	RUNNABLE	4
P2	SLEEPING	1
P3	RUNNABLE	2
P4	RUNNABLE	5

2. Suppose the runnable processes are P1 with priority 3, P2 with priority 1, P3 with priority 1 and P4 with priority 2. Which statement is correct?

a) P4 must run before P2 and P3.
b) P2 and P3 should be preferred over P1 and P4.
c) P1 must always run before P3.
d) All runnable processes are treated identically regardless of priority.

Part B: Scheduler Logic

Consider the following pseudocode skeleton:

```
best = 0;
for each process p in process table:
    if p.state != RUNNABLE:
        continue;
    if best == 0 || _____________________:
        best = p;
```

B.1 Which condition correctly implements priority scheduling?

a) `p->priority > best->priority`
b) `p->priority < best->priority`
c) `p->pid < best->pid`
d) `p->state == RUNNING`

Part C: Required Kernel Modifications

1. Which of the following must be added to the process structure to support priority scheduling?

a) `int priority;`
b) `int tickets;`
c) `char name[16];`
d) `int waiting_time;`

2. Which scheduler behavior must change compared to standard round robin?

a) The scheduler must ignore process states.
b) The scheduler must always choose the first process in the table.
c) The scheduler must scan runnable processes and choose the one with highest priority.
d) The scheduler must put all processes to sleep before selecting one.

Part D: Reading Scheduler Outcomes

Assume lower number means higher priority. The scheduler is invoked repeatedly, and the following processes are always runnable: P1 with priority 2, P2 with priority 0 and P3 with priority 1. Which process should be chosen on the next scheduler invocation?

Part E: Tie Handling

Suppose the runnable processes are: P1 with priority 1, P2 with priority 1 and P3 with priority 3. If ties are broken using round robin, which statement is true?

a) P3 may run before P1 and P2.
b) Only P1 may run.
c) P1 and P2 should alternate over time before P3 is chosen.
d) Priority has no effect.

Part F: Implementation of Priority Scheduling For XV6

Extend XV6 such that the scheduler selects, at each scheduling decision point, the RUNNABLE process with the highest priority. Recall that each process must have an associated integer priority value with a smaller integer value corresponding to a higher priority, and, if multiple `RUNNABLE` processes share the same priority, they must be scheduled using round-robin among themselves. You must: *i*) Extend the process structure (add `priority` field), *ii*) Modify the scheduler to select the `RUNNABLE` process with smallest priority, and *iii*) Ensure round-robin behavior among equal-priority processes. Modify only `proc.h` and `proc.c`. Do not modify locking, process states, or lifecycle. The scheduler must remain preemptive and compatible with timer interrupts and system calls (`fork()`, `exit()`, `wait()`) must not break. Finally, assign a default priority at process creation. Make sure that processes with lower priority values run first and that equal priority processes must alternate execution. In a mixed scenario, higher priority dominates and equal priority processes share CPU fairly. Provide a one-page report detailing your implementation with the full commented code and output logs with screenshots that support your claims of proper operation. If your scheduler does not operate properly, this should also be reflected by your screenshots along with your interpretation of the reason behind the failure.